

DH 308 FINAL PROJECT

Hospital billing integrated with HIS

Prototype build, Chen ER design, API and AWS demo guide

Final deliverables

24-slide PowerPoint, Next.js frontend, FastAPI backend, PostgreSQL schema and this guide.

All patient names, prices, tax settings, payer decisions and documents in the website are invented. Do not enter real patient information.

Prepared 5 October 2026

github.com/sreehariX/tiveri-billing

1. Scope and architecture

The teaching prototype targets a 20 to 50 bed Indian hospital. A local HIS-style event records a delivered service. The billing API prices it, builds a running encounter, and issues one final itemised invoice. Pharmacy dispensing, room stays, packages, advances, simulated claims, receipts, tax review and receivables share one PostgreSQL database.

Implemented	Demo boundary
OPD, IPD, emergency and day-care encounters; self-pay, private TPA, PM-JAY, CGHS and corporate routes.	External HIS, insurer, PM-JAY and GST portal actions are simulated locally.
Payer rates, pharmacy batch stock, tax snapshots, preauth, invoices, claims, receipts, audit and reports.	The site has no login and no production privacy controls. Use invented records only.

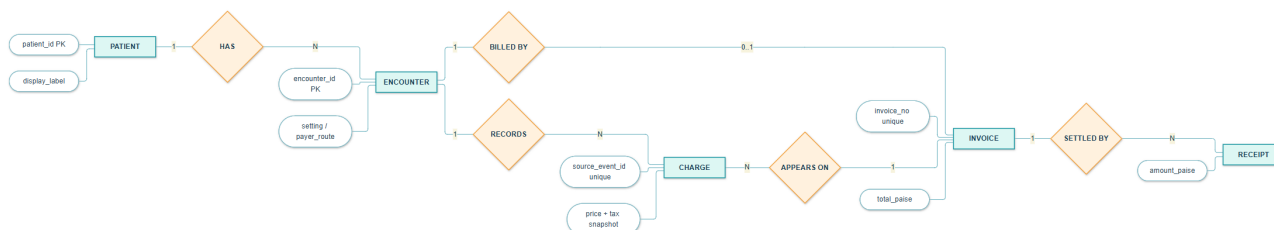
Data flow

- Next.js at port 3000 forwards same-origin /api requests to FastAPI at port 8000.
- FastAPI validates the request and commits the charge, stock, invoice or payer action in PostgreSQL.
- The unique source_event_id makes an identical HIS retry safe. A changed retry returns HTTP 409.
- A charge copies the payer price and effective tax setting. Invoice lines copy them again when finalised.
- A claim decision is separate from a receipt. Only a posted receipt or allocated advance reduces the balance.

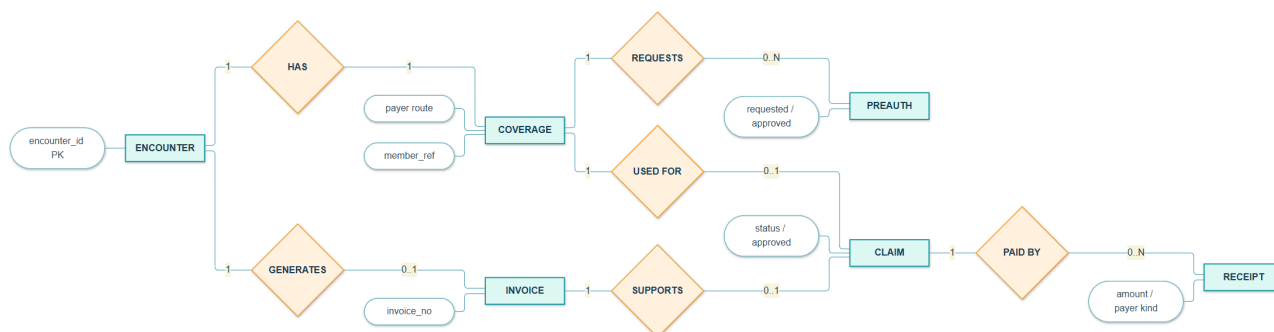
A later authorised exchange could map encounters, charge items, invoices, coverage and claims to relevant FHIR and NRCeS ABDM/NHCX profiles. The present API is a local teaching contract.

2. Chen ER design and schema

Chen notation uses rectangles for entities, diamonds for relationships, ovals for attributes and labels for cardinality. The editable Mermaid source and the rendered images are in docs/core_er.mmd, docs/payer_er.mmd, docs/core_er.png and docs/payer_er.png.



Core diagram. A patient has encounters; an encounter records charges and has at most one final invoice.



Payer diagram. Preauth and claim decisions are different records from receipts.

Tables	Key rule
patients, encounters, coverages	Encounter has patient, setting and payer route.
services, payer_rates, tax_rules	Payer prices and tax versions set future charge values.
stock_batches, dispenses, charges	An unexpired batch is decremented and charged in one transaction.
invoices, invoice_lines	One final invoice per encounter; a charge appears on at most one invoice line.
preauths, claims, advances, receipts	Approval and collection remain separate.

The executable schema is backend/app/schema.sql. FastAPI creates its tables and synthetic seed cases at startup. To change a diagram, edit its .mmd source, keep Chen shapes and cardinalities, render to PNG with Mermaid CLI, then refresh the image in the slide.

3. Backend and API

FastAPI, Pydantic and Pyscopg are in backend/app/main.py. Open /docs on the API server for exact request and response schemas; docs/api_contract.md lists the routes.

Route	Use
POST /api/encounters	Create an invented patient encounter.
POST /api/his/events	Capture one delivered service; retry by event ID.
POST /api/pharmacy/dispense	Decrement an unexpired batch and create its charge.
POST /api/packages/apply	Apply a synthetic package and included lines.
POST /api/preauth and decision	Record a local request and invented payer response.
POST /api/invoices	Finalise one itemised bill.
POST /api/claims and decision	Record a simulated final claim.
POST /api/receipts	Post actual patient or payer money.
GET /api/gstr1 and /api/ar	Read tax review and aged receivables.

HIS event example

```
POST /api/his/events
Content-Type: application/json

{"encounter_id":"E-OPD-01", "service_code":"XRAY",
 "quantity":1, "source_event_id":"CLASS-XRAY-001"}
```

Resending the same payload returns duplicate: true. Changing the quantity while keeping the event ID returns HTTP 409. The frontend sends these calls through forms.

4. Frontend, pharmacy and GST

The Next.js app in frontend/app has Overview, Encounters, Pharmacy, Claims, Tax review, Receivables, Reports, Catalog and Audit views. It calls the API through a same-origin rewrite.

The pharmacy flow checks item, batch, expiry and quantity. It stores item code, HSN/SAC review field, tax category, effective GST rate and charge amount. An unclassified item is blocked from final invoice until reviewed. The seeded MED-A is invented: two units at Rs 100 give Rs 200 base, illustrative 5% GST of Rs 10, and Rs 210

total. This is not a universal rate for medicines.

Qualified clinical healthcare services can be exempt. The CBIC rate table has a conditional 5% entry for specified non-ICU room service above Rs 5,000 per day. Actual medicine and device classification requires item-level review. The GSTR-1 output contains Tables 4, 7, 8, 12 and 13 as a review packet, including exempt supplies and HSN summaries. It is not portal-ready JSON.

5. Local start and checks

Docker Desktop route

```
docker compose up --build
```

Open <http://127.0.0.1:3000>. Compose starts Next.js, FastAPI and PostgreSQL. PostgreSQL is bound only to localhost port 55433. Docker was unavailable on the development computer, so the supplied Compose path was not run there.

Native Windows route used during testing

Install Node.js 24, Python 3.12 and PostgreSQL 16. Create the database `tiveri_demo` and a user allowed to create tables. From the repository root, run terminal 1:

```
python -m venv .venv
.\venv\Scripts\python.exe -m pip install -r backend\requirements.txt
$env:DATABASE_URL = "postgresql://USER:PASSWORD@127.0.0.1:5432/tiveri_demo"
.\venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 127.0.0.1 --
port 8000
```

In terminal 2:

```
cd frontend
npm ci
npm run dev
```

Open <http://127.0.0.1:3000>. API documentation is at <http://127.0.0.1:8000/docs>. Change `API_INTERNAL_URL` before starting Next.js if the API address differs.

Checks and reset

```
curl http://127.0.0.1:8000/api/health
curl -X POST http://127.0.0.1:8000/api/demo/reset
.\venv\Scripts\python.exe -m pytest tests\test_api.py -q
cd frontend
npm run build
```

The database-backed tests cover OPD, TPA, PM-JAY, stock, room, package, payment, refund, tax review and receivables. The website was also exercised in a browser against the live API and PostgreSQL database. Reset only the invented demo data.

6. Live demonstration route

1. Open Overview to show the worklist and outstanding balance.
2. Create a synthetic encounter or select Demo Patient A. Capture a HIS service and inspect the running bill.
3. In Pharmacy, dispense MED-A. Show batch quantity reduction and the taxable charge line.
4. Finalise the self-pay bill and post a patient receipt.
5. For Demo Patient B, record simulated TPA preauth, finalise the IPD bill, submit a claim and decide it. Approval alone leaves a balance. Post a payer receipt.

6. For Demo Patient C, run the simulated PM-JAY check, preauth and package claim. Patient collection is blocked.
7. Open Tax review, Receivables, Reports and Audit. Treat downloaded JSON as a classroom review only.

7. Private AWS EC2 classroom deployment

This is a deployment plan for synthetic data. It was not run on AWS during development. Keep the three application ports bound to loopback and reach the website through an SSH tunnel.

1. Launch an Ubuntu EC2 instance with an SSH key pair and disk space for Docker images and the PostgreSQL volume.
2. In its security group, allow inbound TCP 22 only from My IP. Do not open 3000, 8000 or 55433.
3. Connect with the instance's documented username, commonly ubuntu. Install Docker Engine and Compose using Docker's current Ubuntu guide.
4. Install Git, clone the repository, then start the containers and check the API:

```
sudo apt-get update
sudo apt-get install -y git
git clone https://github.com/sreehariX/tiveri-billing.git
cd tiveri-billing
sudo docker compose up --build -d
sudo docker compose ps
curl http://127.0.0.1:8000/api/health
```

On the presentation laptop, keep this tunnel open and visit <http://127.0.0.1:3000>:

```
ssh -i KEY.pem -L 3000:127.0.0.1:3000 ubuntu@PUBLIC_IP
```

The Compose file binds exposed ports to 127.0.0.1 on the server. The demo password is a training value. Do not enter real patient data. A hospital deployment needs authentication, roles, TLS, secret storage, backups, monitoring, approved tariff and tax masters, and authorised integrations.

8. Future scope and sources

For larger hospitals, add role approvals, multi-location configuration, corporate credit policies, payer document templates, authorised scheme integrations, inventory procurement, reconciliation, migration tools, performance tests and disaster recovery.

The course lectures informed HIS and database context. The friend group's slides informed only visual direction. The supplied ERP Group article suggested workflow topics but was not used as authority for performance statistics or tax rates.

Healthcare and tax	Deployment
CBIC GST rate table	AWS EC2 SSH guide
GST portal GSTR-1 guide	AWS EC2 security groups
NHA PM-JAY operations	Docker Engine for Ubuntu
NHA Health Benefit Packages	Docker Compose plugin
IRDAI health insurance	
NRCeS ABDM FHIR R4	